

Teorie POO din examene

1. Definiți conceptul de **încapsulare** în Programarea Orientată pe Obiecte. Dați un exemplu simplu în JAVA.

Încapsularea este un principiu fundamental al POO care presupune ascunderea detaliilor interne ale unei clase și protejarea datelor de accesul direct din exterior. Se realizează prin:

- Declararea atributelor unei clase ca **private** (acestea nu pot fi accesate direct din afara clasei).
- Furnizarea unor metode **public (getter și setter)** pentru acces controlat la aceste date.

```
Class ContBancar{
    Private double sold;
    //constructor
    //getter
    Public double getSold(){
        Return sold;
    }
}
```

2. Utilizând procedeul de **despachetare** dați un exemplu de secțiune de cod JAVA care reține într-o variabilă de tip primitiv valoarea întreagă împachetată dintr-un obiect instanță a unei clase existente în JAVA.

Despachetarea (unboxing) este procesul prin care un obiect de tip **wrapper class** (ex. Integer, Double, Boolean) este transformat automat într-un tip primitiv (int, double, boolean). Acest proces este gestionat de Java prin **auto-unboxing**, ceea ce înseamnă că nu este nevoie de conversii explicite.

```
Integer obiectInteger=Integer.valueOf(10);
Int valoarePrimitiva=obiectInteger;
```

3. Definiți în JAVA o clasă proprie din care **nu pot fi create instanțe de obiecte**. Pentru a crea o clasă în Java din care nu pot fi create instanțe de obiecte, trebuie să o declarăm **abstractă** sau să îi facem **constructorul privat**.

```
Abstract class ClasaAbstracta
```

4. Dați un exemplu în JAVA de folosire a unei **colecții** în care să rețineți o mulțime de numere reale, astfel încât **să nu poată exista valori duplicate**, iar inserarea/căutarea elementelor în colecție să se facă cât mai eficient posibil.

```
TreeSet sau HashSet
Set<Double> a=new TreeSet<>();
a.add(1.2);
a.add(3.4);
a.add(1.2); //nu va fi adaugat
```

5. Dați un exemplu în JAVA de folosire a unei **colecții** în care să rețineți o mulțime de numere reale, astfel încât **să poată exista valori duplicate**, iar inserarea/căutarea elementelor în colecție să se facă cât mai eficient posibil.

ArrayList

```
ArrayList<Double> a=new ArrayList<>();
```

```
a.add(1.2);
```

```
a.add(3.4);
```

```
a.add(1.2); //poate fi adaugat
```

6. Definiți conceptul de **testare regresivă** și precizați cum poate fi implementată în JAVA.

Testarea regresivă este procesul de **retestare a unei aplicații software după ce au fost făcute modificări**, pentru a ne asigura că **funcționalitățile existente nu au fost afectate negativ**.

În Java, putem face testare regresivă folosind **JUnit**.

7. Definiți conceptul de **polimorfism** în Programarea Orientată pe Obiecte. Dați un exemplu simplu în JAVA.

2 tipuri de polimorfism:

Metodele pot avea **același nume**, dar cu **parametri diferiți**.

Când o **subclasă** suprascrie o metodă definită în **superclasă**.

```
Int aduna(int a,int b){
```

```
Return a+b;
```

```
}
```

```
Int aduna(int a,int b,int c){
```

```
Return a+b+c;
```

```
}
```

```
double aduna(double a,double b){
```

```
return a+b;
```

```
}
```

8. Dați un exemplu în JAVA de o **proprietate specifică fiecărui obiect instanță a unei clase**, care nu poate fi accesată direct decât din clasele aflate în același pachet cu clasa din care fac parte obiectele respective.

Package exemplu;

```
Public class Persoana{
```

```
String nume; //vizibil doar în pachetul exemplu
```

```
}
```

9. Dați un exemplu în JAVA de o **proprietate constantă a fiecărui obiect** care poate fi accesată din orice subclasa a clasei respective

```
Protected final String specie;
```

10. Definiți conceptul de **supraîncărcare a metodelor** în Programarea Orientată pe Obiecte.

Dați un exemplu simplu în JAVA.

Permite definirea **mai multor metode cu același nume**, dar cu **semnături diferite** (adică un număr diferit de parametri sau tipuri diferite de parametri).

```
Int aduna(int a,int b){
```

```
Return a+b;
```

```
double aduna(double a,double b){
```

```
return a+b;
}
```

11. Ambiguitate la supraincercarea metodelor

Metodele **au același nume, dar diferă prin numărul sau tipul parametrilor**.
Ambiguitatea apare atunci când compilatorul **nu poate decide** ce metodă să apeleze, ducând la o eroare de compilare.

```
Void show(int a)
```

```
Void show(long a)
```

12.

13. Dați un exemplu simplu în JAVA de interfață cu o metodă care are **comportament implicit definit**.

Interfețele pot avea metode cu implementare implicită folosind cuvântul cheie **default**.

```
Interface Vehicul{
Default void semnalizeaza(){
System.out.println(„.....”);
}
}
```

14. Dați un exemplu simplu în JAVA de o **funcționalitate care nu poate fi executată simultan** de către două fire de lucru (threads) diferite

Pentru a evita **executarea simultană** a unei funcționalități de către **două fire de execuție (threads)**, folosim **blocuri sincronizate (synchronized)**.

```
Public synchronized void retrageBani()
```

15. Definiți conceptul de **mostenire** în POO. Dați un exemplu simplu în JAVA

Permite unei clase (subclasă) să preia **atributele și metodele** unei alte clase (superclasă).

```
Class Animal{
String nume;
Public void mananca(){
}
Class Caine extends Animal{
Public void latra()
}
}
```

16. Utilizând procedeul de **împachetare** dați un exemplu de secțiune de cod JAVA care reține valoarea reală a unei variabile de tip primitiv într-un obiect instanță a unei clase existente în JAVA.

În **Java**, **împachetarea (Boxing)** este procesul prin care o valoare de **tip primitiv** este **transformată într-un obiect** al clasei corespondente (Wrapper Class).

```
Double valoareReala=12.10;
Double obiectDouble=new Double(valoareReala);
```

17. Definiti in JAVA o **clasa** care **nu poate fi derivate**.

```
final class Calculator
```

18. Definiti conceptul de **jurnalizare** si precitati cum poate fi realizata in JAVA

Jurnalizarea este procesul de **înregistrare** a informațiilor despre executarea unui program, astfel încât să poți urmări **comportamentul aplicației** și să identifici **erorile, evenimentele importante, și statistici relevante**.

Se poate utiliza **java.util.logging**

19. Conceptul de **abstractizare**. Exemplu

Abstractizarea permite **ascunderea detaliilor interne** și furnizarea unei **interfețe clare** pentru utilizatorii obiectului.

```
Abstract class examen{
```

20. Conceptul de **sub-interfata**. Exemplu

O **sub-interfață** este o **interfață care extinde** o altă interfață

```
Interface Vehicul{
```

```
Interface Masina implements Vehicul{
```

21. Exemplu de o clasa cu o metoda instanta care arunca o **exceptie neverificata** si care ulterior este interceptat intr-o alta metoda instanta

```
RuntimeException/NullPointerException
```

```
Public class Exceptie{
Public void metoda1(){
Throws new NullPointerException(„Exceptie Neverificata”);
}
Public void metoda2(){
Try{
Metoda1();
}catch(NullPointerException e){
System.out.println(e.getMessage());
}
}
}
```

22. Exemplu de o clasa cu o metoda non-statica care arunca o **exceptie verificata** si care ulterior este interceptat intr-o alta metoda statica

Try-throw-catch

```
Class Exemplu{
Public void metodaNonStatica() throws IOException{
Throw new IOException(„Eroare la citire”);}
Public static void metodaStatica(){
Exemplu obj=new Exemplu();
Try{
Onj.metodaNonStatica();
}catch (IOException e){
...}
```

23. Exemplificati in JAVA **doua fire de lucru diferite** care incrementeaza **pe rand un acelasi contor** fara ca valoarea contorului sa fie **corupta**.

```
Class Contor{
    Private int valoare=0;
    Public synchronized void incrementeaza(){
        Valoare++;
    }
    Public synchronized int getValoare()
    Return valoare;
}
}
Class FirExecutie extends Thread{
    Private contor contor;
    Public FirExecutie(contor contor){
        This.contor=contor;}

    @Override
    Public void run(){
        For(int i=0;i<10;i++){
            Contor.incrementeaza();
        }
    }
}
```

Main:

```
Contor contor=new Contor();
Thread fir1=new FirExecutie(contor);
Thread fir2=new FirExecutie(contor);
```

```
Fir1.start();
Fir2.start();
```

```
Fir1.join();
Fir2.join();
```

24. Conceptul de **variabila de clasa**. Exemplu

O **variabilă de clasă** este o variabilă **comună tuturor instanțelor** unei clase și este declarată cu modificatorul static. Aceasta înseamnă că **nu aparține unui obiect individual**, ci este partajată de toate obiectele acelei clase.

```
Class Contor{
    Static int contor=0;
    Public void Contor(){
        Contor++;
    }
}
```

25. Variabila instantă

O variabilă de instanță este o variabilă definită într-o clasă, dar în afara metodelor, care este specifică fiecărui obiect (instanță) al clasei.

```
Class Persoana{
String nume;
Public Persoana(String nume){
This.nume=nume;}
}
```

26. Clasa abstractă. Exemplu definire+utilizare

O **clasă abstractă** în Java este o **clasă care nu poate fi instanțiată** și care poate conține metode **abstracte** (fără corp) și metode **obișnuite** (cu implementare).

```
Abstract class Vehicul{
String nume;
//constructor
Abstract void porneste();
}
Class Masina extends Vehicul{
//constructor
@Override
Void porneste(){
System.out.println(„...”);
}
```

27. Conceptul de ascundere a variabilelor în relația de moștenire. Exemplu

Ascunderea variabilelor apare atunci când o **subclasă** definește o **variabilă de instanță** cu același nume ca o variabilă din **superclasă**.

```
Class SuperClasa{
String mesaj=„a”;}
Class SubClasa extends SuperClasa{
String mesaj=„b”;}
}
```

28. Procesul de serializare a unui obiect

Serializarea este procesul prin care un obiect este transformat într-un flux de octeți pentru a fi stocat (într-un fișier, bază de date) sau trimis prin rețea.

```
Class Persoana implements Serializable
```

29. Definirea și crearea unui thread și apoi pornirea execuției lui

```
Class MyRunnable implements Runnable{
@Override
Public void run(){
...
}
}
Main:
Thread t1=new Thread(new MyRunnable());
Thread t2=new Thread(new MyRunnable());
T1.start();
```

```
T2.start();
```

30. Clasa învelitoare

O clasă învelitoare este o clasă care învăluie un tip primitiv într-un obiect.

```
Int num=5;
```

```
Integer wrappedNum=Integer.valueOf(num);
```

31. Agregare

Agregarea reprezintă o relație de tipul "parte-din", dar cu o legătură mai slabă între obiectele implicate. Obiectele pot exista independent unele de altele.

```
Class Motor
```

```
Class Masina{
```

```
Motor motor=new Motor();}
```

32. Instanța

O instanță reprezintă o copie a unei clase create în memorie. Atunci când un obiect este creat, acesta este o instanță a unei clase.

```
Class Car
```

```
Main:
```

```
Car myCar=new Car();
```

33. Compoziție

Compoziția reprezintă o relație mai puternică decât agregarea. Un obiect dintr-o clasă poate conține instanțe ale altor clase și viața acestor obiecte depinde de viața obiectului părinte.

```
Class Motor{}
```

```
Class Masina{
```

```
Private Motor motor;
```

```
Public Masina(){
```

```
Motor=new Motor;
```

```
}
```

```
Main:
```

```
Masina masina=new Masina();
```

34. Fixturi în testarea unitară

Fixturile sunt configurațiile necesare pentru a executa testele unitare. Ele pot include inițializarea obiectelor sau setările necesare pentru testarea unei funcționalități.

```
JUnit
```

35. Asertii

Asertiile sunt folosite în testare pentru a verifica dacă o condiție este adevărată. Dacă condiția este falsă, se va arunca o excepție.

```
Assert(x==5);
```

36. Super-Interfața

Super-interfața se referă la interfața unei clase părinte care este implementată de o subclasă. O interfață poate fi extinsă pentru a crea o ierarhie de interfețe.

```
Interface Animal{
Void sound();}
Interface Mammal extends Animal{
Void walk();}
Class Dog implements Mammal{
Public void sound(){...}
Public void walk(){...}}
```

37. Abstracțiune

Abstracțiunea este procesul de ascundere a complexității interne a unui obiect și de expunere doar a funcționalităților esențiale. În Java, se poate realiza prin clase abstracte sau interfețe.

Abstract class

Interface X

38. Vedere publica si privata

Public înseamnă că variabila sau metoda poate fi accesată din orice altă clasă.

Private înseamnă că variabila sau metoda poate fi accesată doar din cadrul clasei în care este definită.

39. Applet

Un applet este o aplicație Java care rulează într-un **browser web**.